



A hybrid genetic algorithm for the repetition free longest common subsequence problem



Mauro Castelli^{a,*}, Stefano Beretta^{b,c}, Leonardo Vanneschi^{a,b}

^a ISEGI, Universidade Nova de Lisboa, 1070-312, Lisboa, Portugal

^b DISCo, University of Milano Bicocca, 20126 Milano, Italy

^c Inst. for Biomedical Technologies, National Research Council, 20090 Segrate, Italy

ARTICLE INFO

Article history:

Received 11 June 2013

Received in revised form

5 September 2013

Accepted 5 September 2013

Available online 12 September 2013

Keywords:

Repetition free longest common subsequence

Heuristics

Genetic algorithms

Estimation of distribution algorithms

ABSTRACT

Computing the longest common subsequence of two sequences is one of the most studied algorithmic problems. In this work we focus on a particular variant of the problem, called repetition free longest common subsequence (RF-LCS), which has been proved to be NP-hard. We propose a hybrid genetic algorithm, which combines standard genetic algorithms and estimation of distribution algorithms, to solve this problem. An experimental comparison with some well-known approximation algorithms shows the suitability of the proposed technique.

© 2013 Elsevier B.V. All rights reserved.

1. Introduction

One of the most important problem in the field of algorithms on strings is the computation of the *longest common subsequence* (LCS) of two sequences, over a given alphabet. Its importance is mostly due to its wide range of applications, covering many research fields. For example, several applications of different variants of the LCS problem can be found in bioinformatics, which are used to perform analysis on sequences of DNA or RNA. Other interesting applications of variants of the LCS problem can be found in [2]. More specifically, the LCS problem between two given strings S_1 and S_2 asks for finding a longest sequence S which is subsequence of both the input strings S_1 and S_2 (see Section 2 for a formal definition of the problem). Although the LCS problem can be solved in polynomial time (see [14]) using for example a dynamic programming approach (see [7]), its generalization to a set of sequences, which ask for finding the longest sequence that is subsequence of all the input sequences, is NP-hard (see [12]).

Different variants of the longest common subsequence problem have been proposed [4–6,17] to compare biological sequences, where, given two strings S_1 and S_2 , the computed longest common

subsequence is required to satisfy some constraints. An interesting variant of the LCS problem is the so-called *repetition-free longest common subsequence* (RF-LCS) which, given two sequences S_1 and S_2 , asks for finding the longest sequence S that is subsequence of both S_1 and S_2 and such that it contains at most one occurrence of each symbol [1]. This variant is used to model the genome rearrangement with respect to the duplications of some genes. The goal of this study is to infer the (supposed) original sequence of genes, in which every gene has only one occurrence, starting from a set of sequences, each one (possibly) containing multiple copies of some genes [16]. This particular problem has been strongly investigated and the complexity results can be found in [8]. Finally, other recent studies focused on the so-called *doubly-constrained longest common subsequence* (DC-LCS), which generalizes the mathematical formulation of the RF-LCS (and other constrained variants of LCS problems). In particular, some complexity results on this problem can be found in [5], while the work proposed in [3] discusses the parameterized complexity of the RF-LCS problem.

In this paper we define a hybrid *genetic algorithm* (GA) to address the RF-LCS problem. The idea of the proposed technique is to use standard genetic algorithms and estimation of distribution algorithms. In particular a genetic algorithm is used to explore the search space, while an estimation of distribution algorithm provides a simple, efficient and theoretically sound method to guarantee that the constraints of the RF-LCS problem are satisfied.

The paper is organized as follows: Section 2 introduces the basic definitions that will be used in the rest of the paper; Section 3 introduces the genetic algorithms (GAs) describing their

* Corresponding author. Tel.: +351 213828628; fax: +351 213872140.

E-mail addresses: castelli.mauro@gmail.com, mcastelli@isegi.unl.pt (M. Castelli), beretta@disco.unimib.it (S. Beretta), lvanneschi@isegi.unl.pt (L. Vanneschi).

computational model, while Section 4 focuses on the estimation of distribution algorithms (EDAs), presenting their properties and the main differences with GAs. Section 5 presents the hybrid algorithm developed to solve the RF-LCS problem, while Section 6 describes the experimental settings, and discusses the obtained results comparing them with the ones obtained with other approximation algorithms.

2. Basic definitions

In this section we give some basic concepts and notations that will be used in the rest of the paper. Let S be a string over an alphabet Σ of size $|\Sigma|$. Given a string S , we denote by $S[i]$ the symbol occurring at position i in string S . Let also $S[i \dots j]$ be the substring of S starting at position i and ending at position j . Given two sequences S and S' over a finite alphabet Σ , S' is a *subsequence* of S if S' can be obtained from S by removing some (possibly zero) characters. When S' is a subsequence of S , then S is a *supersequence* of S' . Given two sequences S_1 and S_2 , the LCS problem asks for a longest possible sequence S that is a subsequence of both S_1 and S_2 . In the rest of the paper we will focus on the following variant of the LCS problem.

Problem 1 (REPETITION-FREE LONGEST COMMON SUBSEQUENCE (RF-LCS)). **Input:** two strings S_1 and S_2 over an alphabet Σ . **Output:** a longest common subsequence S of S_1 and S_2 so that S contains at most one occurrence of each symbol $\sigma \in \Sigma$.

It has been proved in [1] that Problem 1 is APX-hard even when each symbol occurs at most twice in each of the input strings S_1 and S_2 .

3. Genetic algorithm

Genetic Algorithms (GAs) [9,10] are a class of computational models that mimic the process of natural evolution. GAs are often viewed as function optimizers although the range of problems to which they have been applied is quite broad. Although different variants of GAs exist, most of the methods called “GAs” have at least the following elements in common: populations of chromosomes, selection according to fitness, crossover to produce new offspring, and random mutation of new offspring. The chromosomes in a GA population typically take the form of bit strings. Each locus in the chromosome has two possible alleles: 0 and 1. Each chromosome (or individual) can be thought of as a point in the search space of candidate solutions. The GA processes populations of chromosomes, successively replacing one such population with another. GAs most often require a fitness function that assigns a score (fitness) to each chromosome in the current population. The fitness of a chromosome depends on how well that chromosome solves the problem at hand.

Genetic operators are used to create a new population of chromosomes starting from the existing one (population of parents), and the simplest form of the genetic algorithm involves three types of operators: selection, crossover (single point), and mutation. The selection operator selects chromosomes in the population for reproduction. The fitter the chromosome, the more times it is likely to be selected to reproduce. The crossover operator randomly chooses a locus and exchanges the subsequences before and after that locus between two chromosomes to create two offspring. The crossover operator roughly mimics biological recombination between two single-chromosome (haploid) organisms. The mutation operator randomly flips some of the bits in a chromosome. Mutation can occur at each bit position in a string with some probability, usually very small (e.g., 0.001).

Given a problem to be solved and a bit string representation for candidate solutions, a simple GA works as follows: (1) start with

a randomly generated population of nl -bit chromosomes (candidate solutions to the problem). (2) Calculate the fitness $f(x)$ of each chromosome x in the population. (3) Repeat the following steps until n offspring has been created: (3.1) select a pair of parent chromosomes from the current population, the probability of selection being an increasing function of fitness. Selection is done “with replacement”, meaning that the same chromosome can be selected more than once to become a parent. (3.2) With probability p_c (the “crossover probability”), cross over the pair at a randomly chosen point (chosen with uniform probability) to form two offspring. If no crossover takes place, form two offspring that are exact copies of their respective parents. (3.3) Mutate the two offspring at each locus with probability p_m (the “mutation probability”), and place the resulting chromosomes in the new population. (4) Replace the current population with the new population, and repeat from Step 2 until a certain number of generations have been performed (or other termination criteria are met). Each iteration of this process is called a generation.

This introduction to the basic GA algorithm is taken from [13] and the reader is referred to [13,9] for a complete introduction to GAs.

4. Estimation of distribution algorithm

As observed in [15], from an abstract point of view, the selected set of promising solutions can be viewed as a sample drawn from a probability distribution. Although the true probability distribution is unknown, there are algorithms that are able to estimate that probability distribution by using the selected set of solutions itself and use this estimate to generate new solutions. These algorithms are called *estimation of distribution algorithms* (EDAs) [11]. In EDAs better solutions are selected from an initially randomly generated population of solutions like in the simple GAs. Then, the true probability distribution of the selected set of solutions is estimated and new solutions are generated according to this estimate. The new solutions are then added into the original population, replacing some of the old ones. The process is repeated until the termination criteria are met.

EDAs therefore do the same as GAs except for that they replace genetic crossover and mutation operators by the following two steps: (1) a model (an estimate of the true distribution) of selected promising solutions is constructed and (2) new solutions are generated according to the constructed model. Hence, while in GAs (and other heuristics from evolutionary computation) the interrelations between the different variables representing the individuals are kept in mind implicitly (e.g. building block hypothesis [9]), in EDAs the interrelations are expressed explicitly through the probability distribution associated with the individuals selected at each iteration.

5. Method

In this section, the proposed algorithm used to address the RF-LCS problem is presented. Let S_1 and S_2 be the two sequences over an alphabet Σ that represent the input of the problem. We assume, without the loss of generality, that the two sequences have the same length l . A possible solution for the RF-LCS problem is codified by means of a binary string K that has the same length of the two sequences S_1 and S_2 . More in detail, ‘1’ in a particular position i (with $i \leq l$) indicates that the longest common subsequence (without repetition of characters) between S_1 and S_2 includes the i th character of S_1 . On the other hand, ‘0’ indicates that the common subsequence should not consider the i th character of S_1 . Hence, an individual of the GA population is a string where each bit in position i , with $1 \leq i \leq l$, indicates whether or not the i th character

of S_1 must be considered in the common subsequence between S_1 and S_2 .

Given an initial population P of N chromosomes, the standard GA is used to build a new population of candidate solutions. Anyway, using the standard GA, it is not possible to guarantee that a particular character will appear only once in the common subsequence. To address this problem, we identify two main options: the first one is to penalize the repetition of a particular character c in the common subsequence, integrating this penalty in the fitness function. This approach is not easy because the definition of the fitness function is the key point for a successful use of the GA, and the inclusion of a penalty mechanism makes this definition more tricky and complicated. The second approach, used in the proposed work, is to consider a specific mechanism able to guarantee that all the chromosomes in the population satisfy the constraint of the RF-LCS problem. In particular, the approach used in this paper considers EDA. The idea is to estimate the distribution of promising solutions assuming that the variables in the problem are independent and to look at the values of each variable regardless of the remaining solutions. The model of the selected promising solutions contains a set of frequencies of all values on all string positions in the selected set. The construction of the distribution model is outlined in Algorithm 1.

Algorithm 1: Creation of the frequency vector V

```

Input :  $l$  = length of the chromosomes
         $N$  = number of chromosomes in the population  $P$ 
Output:  $V$  = frequency vector
        // Creates a vector  $V$  of size  $l$ 
1 for  $i \leftarrow 1$  to  $l$  do
2    $V[i] \leftarrow 0$ ;
3 end
   // Selects a subset  $Q$  from  $P$  based on fitness
4 for  $i \leftarrow 1$  to  $l$  do
5   foreach chromosome  $S \in Q$  do
6      $V[i] \leftarrow V[i] + S[i]$ ;
7   end
8 end
9 for  $i \leftarrow 1$  to  $l$  do
10   $V[i] \leftarrow V[i]/N$ ;
11 end
12 return  $V$ ;

```

The vector V of frequencies is then used to resolve conflicts that may occur in the candidate solutions. Considering a string K in the population P , we say that a conflict appears in K if both the following conditions hold: (i) $K[i] = 1, K[j] = 1$ for some i and j such that $1 \leq i, j \leq l$; and (ii) $S_1[i]$ and $S_1[j]$ contain the same symbol. In other words, a conflict appears if and only if the common subsequence contains at least one repetition of one of the characters of the alphabet Σ .

The conflict is solved in the following way: if $V[i] > V[j]$ then $K[j]$ is set to 0. In the opposite case $K[i]$ is set to 0.

To summarize, the final hybrid GA consists of the following steps.

1. Start with a randomly generated population of N l -bit chromosomes (candidate solutions to the problem) where the conflicts are randomly resolved.
2. Calculate the fitness $f(x)$ of each chromosome x in the population.
3. Repeat the following steps until n offspring have been created: (1) select a subset Q of chromosomes from the current population, with the probability of selection being an increasing function of fitness. In this application $|Q| = \frac{|P|}{2}$. (2) Calculate the frequency vector V using Algorithm 1. (3) Update elements in P , resolving all the existing conflicts based on the information

in V . (4) Select (with replacement) a pair of parent chromosomes from P , with the probability of selection being an increasing function of fitness. (5) With probability p_c (the “crossover probability”), cross over the pair of selected parents at a randomly chosen point (chosen with uniform probability) to form two offspring. If no crossover takes place, form two offspring that are exact copies of their respective parents. (6) Mutate the two offspring at each locus with probability p_m (the “mutation probability”), and place the resulting chromosomes in the new population.

4. Replace the current population with the new population, and repeat from Step 2 until a certain number of generations has been performed (or other termination criteria are met).

6. Experimental study

In this section the experimental phase is described. In order to compare the performances obtained by the proposed hybrid GA, three existing algorithms designed to address the RF-LCS problem have been considered. The three algorithms are presented in [1], and Section 6.2 reports a brief description of the three algorithms. Using the same notation defined in [1], from now on the three algorithms used as benchmarks will be referred as A1, A2 and A3. The proposed hybrid GA is tested against the three approximation algorithms on randomly generated instances of the RF-LCS problem. In particular, two parameters have been considered: the length of the sequences and the alphabet size as a function of the length. Each position of a randomly generated sequence is one of the symbols of the alphabet chosen uniformly at random. In these sequences, most of the symbols have approximately the same number of occurrences.

This section is organized as follows: Section 6.1 outlines the parameters of the GA and describes how experiments have been performed. Section 6.2 reports a brief description of the three algorithms proposed in [1], while Section 6.3 discusses the obtained results, comparing them with the ones obtained with algorithms A1, A2 and A3.

6.1. Settings

In this section the experimental setting is described. A total of 30 runs of the proposed hybrid GA have been performed for each considered combination of sequence length and alphabet size. Regarding the sequence lengths (denoted with l), 5 different values (32, 64, 128, 256, 512) have been considered and, for each of these values, 7 different alphabet sizes equal to $\frac{l}{8}, \frac{l}{4}, \frac{3l}{8}, \frac{l}{2}, \frac{5l}{8}, \frac{3l}{4}, \frac{7l}{8}$ have been considered. Hence a total of 1050 runs have been performed. All the runs used populations of 100 individuals and the evolution stopped after 1000 generations. Candidate solutions are represented as binary strings of length equal to the length of the sequences. Individuals are randomly generated, with a probability of having 0 or 1 in a specific locus equal to 0.5. To create new individuals, one point crossover and flip mutation with probabilities equal to 0.9 and 0.05, respectively, have been used. To determine the values of the GA's parameters, a preliminary experimental study has been performed for finding the parameter setting able to return the best results. Survival from one generation to the other was always guaranteed to the best individual of the population (elitism).

As already explained, an individual K of the GA population is a string where each bit $1 \leq i \leq l$ indicates whether or not the i th character of S_1 must be considered in the common subsequence between S_1 and S_2 . Let S'_1 be the substring obtained from S_1 considering only the characters $S_1[i]$ such that $K[i] = 1$ (with $1 \leq i \leq l$). The fitness of an individual is defined as the length of

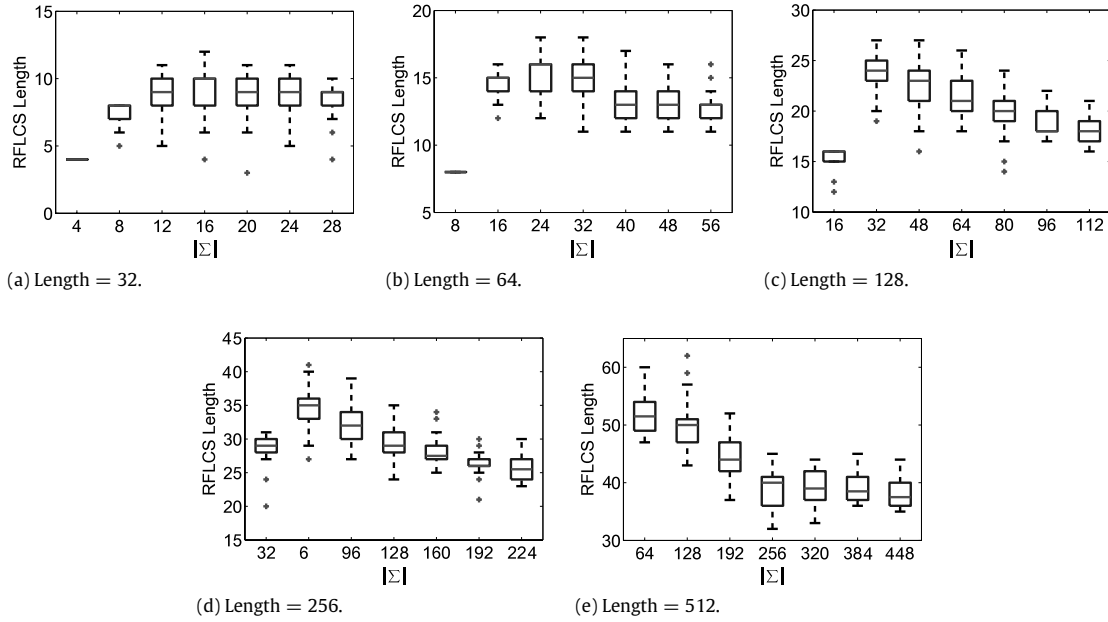


Fig. 1. (a) Length of RF-LCS for strings length equal to 32 and different alphabet size. (b) Length of RF-LCS for strings length equal to 64 and different alphabet size. (c) Length of RF-LCS for strings length equal to 128 and different alphabet size. (d) Length of RF-LCS for strings length equal to 256 and different alphabet size. (e) Length of RF-LCS for strings length equal to 512 and different alphabet size.

the LCS between S'_1 and S_2 penalized by a factor that considers the number of characters in S'_1 that are not in the LCS between S'_1 and S_2

$$f(k) = LCS(S'_1, S_2) - \sum_{j=1}^{len(S'_1)} \phi(S'_1[j], S_2) \quad (1)$$

where

$$\phi(S'_1[j], S_2) \leftarrow \begin{cases} 1 & \text{if } S'_1[j] \notin LCS(S'_1, S_2) \\ 0 & \text{in the opposite case.} \end{cases} \quad (2)$$

It is important to underline that the LCS between S'_1 and S_2 also corresponds to an RF-LCS between the same strings, because after resolving the conflicts in K (see Section 5), $S'_1[i] \neq S'_1[j]$ for all i and j such that $1 \leq i, j \leq len(S'_1)$ with $len(S'_1)$ denoting the length of S'_1 .

6.2. Existing approximation algorithms

In this section, algorithms A1, A2, A3 are briefly described. These algorithms have been considered to compare the results of the proposed hybrid GA algorithm with the ones of existing approximation algorithms for the RF-LCS problem. For all the details the reader is referred to [1].

For each symbol a and a sequence S , we define $n(S, a)$ as the number of appearances of a in S and $m_a(S_1, S_2) = \min\{n(S_1, a), n(S_2, a)\}$.

Algorithm A1 consists of the following steps: given sequences S_1 and S_2 , compute an LCS of S_1 and S_2 and remove all repeated symbols but one, in the obtained LCS. Return the resulting sequence. Let m be the maximum value of $m_a(S_1, S_2)$ taken over all a . It is not hard to see that Algorithm A1 is an m -approximation for the RF-LCS problem between S_1 and S_2 .

Algorithm A2 is probabilistic. It consists of the following steps: given sequences S_1 and S_2 , for each symbol a , if $m_a(S_1, S_2) = n(S_1, a)$, pick uniformly at random one of the $m_a(S_1, S_2)$ occurrences of a in S_1 , and delete all the others from S_1 ; if $m_a(S_1, S_2) \neq n(S_1, a)$, pick uniformly at random one of the $m_a(S_1, S_2)$ occurrences of a in S_2 , and delete all the others from S_2 . Let S'_1 and S'_2 be the resulting sequences after this clean-up. Compute an LCS S' of S'_1 and S'_2 and return S' .

Algorithm A3 is a variant of Algorithm A2 that uses less random bits. It works basically as Algorithm A2 in the sense that, for each repeated symbol a , the sequence (either S_1 or S_2) that is chosen to keep an occurrence of a is the one with the least number of repetitions of a . The difference now is that A3 picks uniformly at random only one number, say r , in the interval $[0; 1]$ and uses r to select which occurrence of each symbol will remain. For each repeated symbol, the selection is done using the same number r . (If a symbol a occurs k times in the chosen sequence, the interval $[0; 1]$ is partitioned into k subintervals of the same size, each one corresponding to an occurrence of a , and then the occurrence of a that corresponds to the subinterval that contains r is kept.)

6.3. Results

This section describes the obtained results. First of all, the results produced by the proposed algorithm are presented and then, in the second part of this section, those results are compared with the ones obtained with the algorithms proposed in [1].

Box plots in Fig. 1 report the length of the RF-LCS found by the hybrid GA, for different sequence lengths and different alphabet sizes. It is possible to analyze these box plots considering a certain sequence length and analyzing how the performances vary when different alphabet sizes ($|\Sigma|$) have been considered. For sequence lengths equal to 32 and 64 (Fig. 1(a) and (b)) it is possible to see that the proposed algorithm always finds the optimal LCS in all the performed runs for $|\Sigma| = 1/8$. Also considering the remaining box plots (Fig. 1(c)–(e)) it is possible to see that for $|\Sigma| = 1/8$ the hybrid-GA is able to find near-optimal solutions. Considering the remaining alphabet sizes, for the sequence length of 32 there are no significant differences between the different values of $|\Sigma|$. The situation slightly changes in Fig. 1(b), where a sequence length of 64 has been considered. In this case, the performances of the algorithm start to degrade for increasing values of $|\Sigma|$. This trend (degradation of the performances of the algorithm for increasing values of $|\Sigma|$) is more evident for the sequence lengths of 128, 256, 512. In this last case the degradation of the performances is even more evident. In particular, Fig. 1(e) suggests the existence of a critical threshold for the value of $|\Sigma|$: when value of $|\Sigma|$ is greater

Table 1

Comparison between the proposed hybrid GA and the methods reported in [1]. For the GA column the average length of the RF-LCS over 30 independent runs is reported. For the other columns, results presented in [1] have been reported. Light gray cells represent the best values obtained for a specific combination of chromosome length and alphabet size. Bold values indicate that the results are statistically better than the other techniques. The Wilcoxon rank-sum test (with a Bonferroni correction for the value of α) for pairwise data comparison has been used under the alternative hypothesis that the samples do not have equal medians.

$ \Sigma $	l	GA	A1	A2	A3
$l/8$	32	4	4	4	4
	64	8	7.8	8	7.9
	128	15.1	15.3	15.7	14.2
	256	27.9	25.8	23.1	21.3
	512	51.7	52.1	40.5	36.5
$l/4$	32	7.6	6.5	7.2	6.9
	64	14.4	12.7	13.9	12.9
	128	22.6	21.7	20.5	19.2
	256	34.7	36.2	31	28.9
	512	50.2	58.2	46.2	43.2
$3l/8$	32	8.9	7.8	8.7	7.8
	64	15.5	13.9	14.7	13.3
	128	22.3	22.5	21.9	20.6
	256	32.2	35.7	31.6	30.3
	512	44.2	53.7	44.9	43.3
$l/2$	32	9.1	8.2	8.6	7.9
	64	14.9	13	13.9	12.7
	128	21.4	21.3	21	19.6
	256	29.3	33.5	30.7	29.3
	512	38.9	50.3	44.7	42.3
$5l/8$	32	8.7	7.6	7.8	7.5
	64	13.2	12.8	12.9	12.5
	128	19.8	20.4	19.8	19.3
	256	28.1	31.5	29.6	27.9
	512	39.3	46.2	42.4	41.3
$3l/4$	32	8.8	6.7	7.6	7.1
	64	13	11.9	12.5	11.9
	128	18.7	19.4	19.2	18.1
	256	26.3	28.4	28	26.9
	512	39	42.5	39.8	39.4
$7l/8$	32	8.4	7.2	7.4	7.1
	64	12.7	11.6	11.8	11.3
	128	18.4	18.4	18.4	17.9
	256	25.6	26.8	26	25.2
	512	38	39.2	37.4	36.6

than 256, the obtained results seem to be independent from the particular alphabet size value. To summarize, results reported in Fig. 1 seem to suggest that the proposed hybrid-GA produces the best performances for low values of both the considered parameters. In order to have a much clear indication about the quality of the results achieved with the proposed algorithm, it is fundamental to perform a comparison with the results obtained with the algorithms presented in [1]. Table 1 reports the average length of the RF-LCS obtained over 30 independent runs of hybrid GA and the results obtained with the techniques proposed in [1]. From this table it is possible to notice that the proposed GA-based method and the technique denoted by A1 are the best performers. More in

detail, the proposed method produces the best results for “small” values (32, 64 and 128) of the sequence length parameter. On the other hand, results reported in Table 1 show that A1 is the best performer when greater values of sequence length parameter (equal or up to 128) have been considered. The other considered techniques (A2 and A3) are not able to outperform A1 and the GA-based method, but it is possible to observe their general behavior. A2 performs better than A1 for values of the sequence length parameter smaller or equal to 64, independently from the considered alphabet size. Regarding A2, it is also possible to note that it produces better performances than the proposed GA-based method for sequence lengths greater than 128. A3 is the worse technique among the ones considered, but it is able to outperform the GA-based technique for sequence lengths of 512 with alphabet size values of $\frac{l}{2}$, $\frac{5l}{8}$, $\frac{3l}{4}$.

6.4. Analysis of the algorithms

In this section we analyze the efficiency of the proposed algorithm with respect to the ones reported in [1]. As authors of [1] pointed out, from a theoretical point of view, all 3 considered algorithms have the same approximation ratio. More precisely, let $m_a(x, y)$ be the minimum between the numbers of occurrences of symbol $a \in \Sigma$ in x and in y and let m be the maximum value of $m_a(x, y)$ taken over all $a \in \Sigma$ (i.e. $m = \max_{a \in \Sigma} m_a(x, y)$). Consider the general case in which the number of symbols a for which $m_a(x, y)$ is bounded by a constant k . Then the algorithms A1, A2 and A3 are m -approximations of RF-LCS. Moreover, these approximation algorithms are obviously polynomial in time because they require the computation of a LCS between two strings (which is polynomially solvable) plus a processing (before or after the LCS computation). In fact, all these algorithms perform a pre- or post-processing of the strings which depends on the maximum number of occurrences of symbols in the strings. For this reason (and under the assumption in which the number of repetitions of symbol a , i.e. $m_a(x, y)$, is bounded by a constant) pre- or post-processing step requires $O(l)$, that must be added to the time to solve an instance of the LCS problem (which requires $O(l^2)$). This means that the overall time is linear. Finally, as shown by the authors, under the same assumption for which $m_a(x, y) < k$, also by considering all the possible substrings x' (resp. y') obtained by keeping one occurrence of each repeated symbol of the input string x (resp. y), the computation of the LCS among every possible combination of x' and y' requires to solve $O(l^k)$ different LCS instances, which is still polynomial.

Regarding the proposed hybrid GA, we have a polynomial time $O(l^2)$ to calculate the fitness and a linear time $O(l)$ to calculate the distribution used by the EDA. But, differently from the three approximation algorithms presented in [1], the execution time must consider a multiplicative constant $P \cdot G$ (where P denotes the number of solutions in the GA population and G the number of iterations of the algorithm) that is typical of all the evolutionary algorithms. Hence, the time needed to solve an instance of the RF-LCS with the proposed algorithm is $P \cdot G \cdot O(l^2)$, which is linear in l .

References

- [1] S.S. Adi, M.D. Braga, C.G. Fernandes, C.E. Ferreira, F.V. Martinez, M.F. Sagot, M.A. Stefanec, C. Tjandraatmadja, Y. Wakabayashi, Repetition-free longest common subsequence, *Discrete Applied Mathematics* 158 (2010) 1315–1324.
- [2] L. Bergroth, H. Hakonen, T. Raita, A survey of longest common subsequence algorithms, in: *Proceedings of the 7th International Symposium on String Processing Information Retrieval, SPIRE '00*, IEEE Computer Society, 2000, pp. 39–48.
- [3] G. Blin, P. Bonizzoni, R. Dondi, F. Sikora, On the parameterized complexity of the repetition free longest common subsequence problem, *Information Processing Letters* 112 (2012) 272–276.

- [4] P. Bonizzoni, G. Della Vedova, R. Dondi, G. Fertin, R. Rizzi, S. Vialette, Exemplar longest common subsequence, *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 4 (2007) 535–543.
- [5] P. Bonizzoni, G. Della Vedova, R. Dondi, Y. Pirola, Variants of constrained longest common subsequence, *Information Processing Letters* 110 (2010) 877–881.
- [6] F.Y. Chin, A.D. Santis, A.L. Ferrara, N. Ho, S. Kim, A simple algorithm for the constrained sequence problems, *Information Processing Letters* 90 (2004) 175–179.
- [7] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, *Introduction to Algorithms*, second ed., MIT Press, 2001.
- [8] C.G. Fernandes, C.E. Ferreira, C. Tjandraatmadja, Y. Wakabayashi, A polyhedral investigation of the lcs problem and a repetition-free variant, in: *Proceedings of the 8th Latin American Conference on Theoretical Informatics, LATIN'08*, Springer-Verlag, 2008, pp. 329–338.
- [9] D. Goldberg, *Genetic algorithms in search, optimization, and machine learning*, in: *Artificial Intelligence*, Addison-Wesley, 1989.
- [10] J.H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, 1992.
- [11] P. Larrañaga, *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*, Kluwer Academic Publishers, 2002.
- [12] D. Maier, The complexity of some problems on subsequences and supersequences, *Journal of the ACM* 25 (1978) 322–336.
- [13] M. Mitchell, *An Introduction to Genetic Algorithms*, in: *Complex Adaptive Systems*, MIT Press, 1998.
- [14] M. Paterson, V. Dancík, Longest common subsequences, in: *Proceedings of the 19th International Symposium on Mathematical Foundations of Computer Science, MFCS'94*, Springer, 1994, pp. 127–142.
- [15] M. Pelikan, D.E. Goldberg, F.G. Lobo, A survey of optimization by building and using probabilistic models, *Computational Optimization and Applications* 21 (2002) 5–20.
- [16] D. Sankoff, Genome rearrangement with gene families, *Bioinformatics* 15 (1999) 909–917.
- [17] Y.T. Tsai, The constrained longest common subsequence problem, *Information Processing Letters* 88 (2003) 173–176.